

# Topology-Aware Deep Reinforcement Learning for Multi-Hop URLLC in DECT-2020 NR Mesh Networks

Benjamin Rauwolf  
Institut für Kommunikationstechnik  
Leibniz Universität Hannover  
Hanover, Germany  
b.rauwolf@outlook.de

Awais Bin Asif  
Institut für Kommunikationstechnik  
Leibniz Universität Hannover  
Hanover, Germany  
asif@ikt.uni-hannover.de

Jürgen Peissig  
Institut für Kommunikationstechnik  
Leibniz Universität Hannover  
Hanover, Germany  
peissig@ikt.uni-hannover.de

**Abstract**—DECT-2020 New Radio (NR) is a radio interface technology (RIT) standardized by the European Telecommunications Standards Institute (ETSI) for large-scale massive machine-type communication (mMTC) and Ultra-Reliable and Low-Latency Communications (URLLC) deployments. In DECT-2020 NR mesh networks, multi-hop delivery is typically realized by flooding-based forwarding, which is robust but increases network load and does not explicitly optimize end-to-end latency or reliability. In this paper, we propose a topology-aware Deep Reinforcement Learning (DRL) framework for URLLC-oriented multi-hop routing in DECT-2020 NR mesh networks. Next-hop selection is modeled as a Markov decision process (MDP) whose state combines link-level latency and packet error rate (PER) metrics with graph-structured topology information. An inductive graph neural network (GNN) encoder summarizes the current mesh topology and a per-edge Deep Q-Network (DQN) head selects the next hop using a URLLC-shaped reward that jointly captures end-to-end latency and reliability. To improve scalability, the agent is trained through a curriculum over progressively larger topologies. Simulation results over small to large mesh topologies show that the proposed GNN+DRL routing policy improves the URLLC trade-off compared with both the existing latency-only routing method and the DECT-2020 NR flooding procedure. For a representative medium-sized mesh network, it reduces end-to-end latency by up to 19.9%, lowers end-to-end PER by up to 10%, and decreases the number of transmissions per delivered packet compared with the existing approach.

**Index Terms**—DECT-2020 NR, URLLC, routing, deep reinforcement learning, graph neural networks, mesh networks.

## I. INTRODUCTION

Digital Enhanced Cordless Telecommunications (DECT), standardized by European Telecommunications Standards Institute (ETSI), has evolved into Digital Enhanced Cordless Telecommunications-2020 New Radio (DECT-2020 NR) as a radio interface technology tailored for large-scale, low-power Internet of Things (IoT) and industrial scenarios. Operating in license-exempt spectrum with decentralized scheduling and mesh support, DECT-2020 NR enables self-organizing multi-hop topologies in which battery-powered radio devices (RDs) may operate in Fixed Termination (FT), Portable Termination (PT), or combined FT/PT modes. The corresponding physical-

layer, Medium Access Control Layer (MAC)-layer, and data-link/convergence-layer procedures are specified in a series of ETSI standards [1]–[4].

In the standardized DECT-2020 NR system, multi-hop communication is typically realized via flooding at the data-link layer [4]. Flooding is robust against link failures and topology changes, but it also causes redundant transmissions, higher interference, and increased resource consumption in dense deployments [5]. Deterministic routing based only on hop count or latency can reduce this overhead, but such schemes do not explicitly capture time-varying packet errors, interference, and the coupled reliability-latency constraints that are central to ultra-reliable and low latency communication (URLLC) [6]. Consequently, efficient multi-hop communication under end-to-end URLLC constraints remains challenging in dense industrial IoT environments.

Recent work on DECT-2020 NR has mainly focused on standard evaluation, link-level characterization, and latency-oriented routing. Anttonen et al. report system-level performance for dense massive machine-type communication (mMTC) deployments [7], while Kovalchukov et al. summarize the overall DECT-2020 NR design and positioning as a 5G mMTC technology [5]. ETSI evaluation and standardization documents characterize link-level and system-level latency and reliability under multiple deployment scenarios [1]–[4], [8]. Measurement and simulation studies further analyze practical link behavior via testbed-based measurements [9] and an open-source link-level simulator [10]. More recent work has started to address routing-related optimization: [11] proposes a MAC-layer mechanism to reduce outage probability in indoor industrial scenarios, and [6] uses disseminated link metrics together with Dijkstra’s shortest-path algorithm to minimize end-to-end latency instead of relying on flooding.

In parallel, learning-based wireless networking has advanced rapidly in the last few years [12]. Deep Reinforcement Learning (DRL) has shown promise for routing and sequential decision problems under uncertainty [13]–[15], while recent surveys highlight Graph Neural Networks (GNNs) as a natural model class for wireless and graph-structured communication

problems because they encode topology, link features, and neighbourhood context directly [16]–[18]. Recent routing-oriented combinations of GNNs and DRL further indicate that topology-aware learning can generalize across unseen graphs and capture routing objectives beyond shortest-path latency [19].

Motivated by these observations, we propose a topology-aware GNN+DRL multi-hop routing framework for dynamic DECT-2020 NR mesh networks that explicitly targets the URLLC reliability-latency trade-off. The gateway maintains a time-varying network graph whose edge costs are defined by user-plane latency  $L_{ij}$  and packet error rate  $p_{ij}$ , consistent with the DECT-2020 NR system model [3] and ETSI evaluation methodology [8]. RDs operating in FT/PT mode disseminate these link metrics to the gateway via existing MAC and convergence-layer signaling, extending the latency-centric graph construction of [6]. The gateway then executes a GNN encoder and a per-edge Deep Q-Network (DQN) policy to compute topology-aware next-hop decisions. For downlink transmission, routing information is multiplexed into MAC Service Data Units (SDUs) and packets are forwarded hop-by-hop using unicast MAC headers, thereby avoiding the standard flooding procedure. Compared with [6], the proposed framework preserves the latency-awareness of centralized path computation while explicitly incorporating reliability and scaling to larger, time-varying mesh topologies.

The remainder of this paper is organized as follows: Section II presents the system model and problem formulation. Section III outlines the proposed GNN-enhanced DRL framework. Section IV describes the simulator and evaluation parameters. Section V discusses the performance results. Section VI concludes the paper.

## II. SYSTEM MODEL AND PROBLEM FORMULATION

We consider an industrial indoor DECT-2020 NR mesh deployment for URLLC, represented by an undirected graph  $G = (V, E)$ , where  $V = \{v_1, v_2, \dots, v_n\}$  denotes the set of DECT-2020 NR RDs and  $E \subseteq V \times V$  denotes the set of wireless links. The topology contains one gateway node that provides backhaul access and acts as the downlink traffic source, while the remaining RDs operate in FT, PT, or combined FT/PT mode. Combined FT/PT nodes can relay traffic and therefore enable multi-hop connectivity and self-organization.

For each link  $e = (i, j) \in E$ , we model a user-plane latency  $L_{ij}$  and a packet error rate  $p_{ij}$ . Following the ETSI evaluation methodology for frame-based DECT-2020 NR operation [8], the single-hop latency without hybrid automatic repeat request (HARQ) is

$$L_{ij} = t_{sym} + t_{rd\_tx} + t_{FA\_switch} + t_{dur} + t_{rd\_rx}, \quad (1)$$

where  $t_{sym}$  is the symbol alignment time,  $t_{rd\_tx}$  is the transmitter processing delay,  $t_{FA\_switch}$  is the frame-alignment and time-division duplex (TDD) switching time,  $t_{rd\_rx}$  is the

receiver processing delay, and  $t_{dur}$  is the packet transmission duration,

$$t_{dur} = \left\lceil \frac{P_s \cdot 8}{R_{ij}} \right\rceil T_{sub}^{ij}, \quad (2)$$

with packet size  $P_s$  in bytes, link throughput  $R_{ij}$  in bit/s determined by the selected modulation and coding scheme (MCS), and subslot length  $T_{sub}^{ij}$  determined by the supported DECT-2020 NR transmission configuration  $(\mu, \beta)$  [3]. The packet error rate, denoted as  $p_{ij}$ , depends on the signal-to-interference-plus-noise ratio (SINR) and the selected MCS [1], [5]. In the simulator,  $p_{ij}$  is obtained from a SINR-to-PER mapping derived from our DECT-2020 NR link-level simulations [10]. Operationally, we define

$$p_{ij} = \frac{n_d}{n_t}, \quad (3)$$

where  $n_d$  is the number of corrupted packet receptions and  $n_t$  is the number of transmitted packets observed for link  $(i, j)$ .

Let  $S \in V$  denote the gateway, let  $T \in V$  denote the destination node, and let  $\Pi(S, T)$  be the set of paths from  $S$  to  $T$ . For a candidate path  $\mathcal{P} \in \Pi(S, T)$ , the end-to-end latency and end-to-end success probability are expressed as follows:

$$L_{\mathcal{P}} = \sum_{(i,j) \in \mathcal{P}} L_{ij}, \quad (4)$$

$$P_{\mathcal{P}} = \prod_{(i,j) \in \mathcal{P}} (1 - p_{ij}). \quad (5)$$

The routing objective is to find a path that minimizes a weighted reliability-latency cost while satisfying URLLC constraints:

$$\begin{aligned} \min. \quad & \sum_{(i,j) \in E} \left( w_L L_{ij} - w_p \log(1 - p_{ij}) \right) x_{ij} \\ \text{s.t.} \quad & \sum_{(i,j) \in E} L_{ij} x_{ij} \leq L_{\max} \\ & \sum_{(i,j) \in E} \log(1 - p_{ij}) x_{ij} \geq p_{\min} \\ & \sum_{(i,j) \in E} x_{ij} = 1, \quad i \in S, \\ & \sum_{(i,j) \in E} x_{ij} = 1, \quad j \in T, \\ & \sum_{(i,j) \in E} x_{ij} = \sum_{(j,u) \in E} x_{ju} \quad \forall j \in V - \{S, T\} \\ & c_{ij} \geq x_{ij} \quad \forall (i, j) \in E, \\ & x_{ij} \in \{0, 1\} \quad \forall (i, j) \in E, \end{aligned} \quad (6)$$

where  $w_L$  and  $w_p$  balance latency and reliability, while  $L_{\max}$  and  $p_{\min}$  denote the target end-to-end latency and reliability requirements. Moreover  $E$  is the set of edges in the network and  $x_{ij}$  is the binary decision variable of the optimization problem, which represents whether a link  $(i, j)$  is selected or not. If the link  $(i, j)$  is selected for packet transmission, the value of  $x_{ij}$  is set to 1; otherwise, the value of  $x_{ij}$  is set to 0.

The constraints in equation (6) ensure that there should be one neighbour node connected to the source node  $S$  (i.e., gateway) to transmit data packets. Similarly, for the destination node  $T$ , there should be one node connected to receive the data packets. The other constraints ensure the loop-free intermediate links to deliver the packets successfully to the destination. The existence of links in the network is ensured by the binary constant  $c_{ij} \in \{0, 1\}$ . Equation (6) also inspired the hop-wise reward in the proposed learning framework, which keeps the optimization objective and the learned policy aligned.

### III. GNN-ENHANCED DRL FRAMEWORK

#### A. Overview

The proposed framework follows a closed-loop control pipeline. First, each RD estimates per-link cost metrics to its neighbours, namely user-plane latency  $L_{ij}$  and packet error rate  $PER_{ij}$ . These metrics are periodically (or event-triggered) disseminated to the gateway by piggybacking them in DECT-2020 NR MAC and convergence-layer signaling using the MAC multiplexing header, such that control information and user data can be carried within the same MAC protocol data unit (PDU) [2], [6]. Second, the gateway aggregates the reported metrics and constructs a weighted, time-varying network graph  $G_t = (V, E)$  whose edge attributes are  $\{L_{ij}, p_{ij}\}$ , yielding a global topology view consistent with the DECT-2020 NR system model and ETSI evaluation report on DECT-2020 NR [3], [8]. Third, given  $G_t$ , the gateway executes the GNN encoder to obtain topology-aware node embeddings and applies the per-edge DQN head to score feasible next-hop candidates, thereby computing hop-by-hop forwarding decisions (equivalently, an end-to-end route) that optimize the URLLC-shaped objective. Finally, the selected route is used for data delivery: the gateway embeds the routing information inside the packets via DECT-2020 NR control signaling (e.g., convergence-layer/DCCH SDUs) that is multiplexed into MAC PDUs using the MAC MUX header [2] and packets are forwarded hop-by-hop using unicast transmissions along the chosen next hops, avoiding flooding and reducing redundant transmissions and network load. In the remainder of this paper, whenever an edge is written as  $e = (i, j)$ , we use the shorthand notation  $L_e \equiv L_{ij}$  and  $p_e \equiv p_{ij}$ .

#### B. Proposed Framework

We model multi-hop routing as an episodic finite-horizon standard Markov decision process (MDP)  $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ , where  $\mathcal{S}$  denote state,  $\mathcal{A}$  denote actions,  $P$  represents the transition probability,  $R$  represents reward, and  $\gamma$  is the discount factor. An episode corresponds to forwarding one packet from a source node  $S$  to a target node  $T$  over a time-varying graph. At each decision step  $t$ , the agent selects the next hop based on local link metrics (i.e., latency and packet error rate (PER)) and the topology context encoded by the GNN.

This formulation follows the standard Reinforcement Learning (RL) element-wise description (i.e., agents, states, actions, reward, and policy) as shown in Fig. 1.

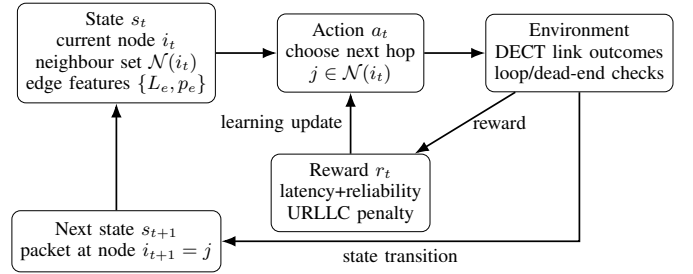


Fig. 1: MDP abstraction for next-hop routing: the agent selects a neighbour, observes the resulting transition, and receives a URLLC-shaped reward.

1) *Agent*: The routing agent determines the forwarding decision at each hop. In our implementation, the learned policy is executed at the controller/gateway that has access to the current network graph and disseminated link metrics, and it outputs next-hop decisions (equivalently, an end-to-end route) for packet forwarding.

2) *State*: At decision step  $t$ , the packet resides at node  $i_t$ . The state  $s_t$  captures the local decision context: (i) the current node index  $i_t$  (and optionally the destination index), (ii) an ordered list of neighbour indices  $j \in \mathcal{N}(i_t)$ , and (iii) a per-neighbour feature matrix denoted as  $\mathbf{X}_t \in \mathbb{R}^{|\mathcal{N}(i_t)| \times 2}$  with rows  $[L_e, p_e]$  for candidate edges  $e = (i_t, j)$ . In addition, topology-aware node embeddings produced by the GNN encoder are exploited as part of the state representation.

3) *Action*: The action  $a_t$  is the selection of a next-hop neighbour  $j \in \mathcal{N}(i_t)$ . We restrict actions to feasible neighbours (connectivity) and enforce hard per-hop bounds by masking actions that violate the URLLC constraints, i.e., maximum allowable latency and PER thresholds.

4) *Transition*: After selecting  $a_t = j$ , the packet is forwarded over link  $(i_t, j)$  and the next state becomes  $i_{t+1} = j$ . The episode terminates when the packet reaches the destination (success) or when forwarding becomes invalid, e.g., due to a dead-end (no feasible neighbour) or loop detection (revisiting nodes).

5) *Reward*: To align hop-wise decisions with the URLLC latency-reliability objective, we define a shaped per-step reward that penalizes both latency and reliability violation. The reward for the chosen edge  $e$  at step  $t$  is expressed as follows:

$$r_t = - \underbrace{w_p \cdot \log(1 - p_e)}_{\text{reliability cost}} - \underbrace{w_L \cdot L_e}_{\text{latency cost}} - \underbrace{r_{\text{urllc}}}_{\text{URLLC cost}}, \quad (7)$$

where  $w_p$  and  $w_L$  weight reliability and latency, respectively and are tuned to reflect the URLLC requirements. The conditional penalty term is defined as follows:

$$r_{\text{urllc}} = \begin{cases} 0, & \text{URLLC bounds not violated,} \\ c_{\text{urllc}}, & \text{latency or PER bound violated,} \\ 2c_{\text{urllc}}, & \text{both bounds violated.} \end{cases} \quad (8)$$

where  $c_{\text{urllc}}$  is a large positive value. The agent maximizes its reward and  $r_{\text{urllc}}$  ensures URLLC compliance by applying

a "soft shield", contrary to "hard shielding", where violating links would be flat out banned from selection. If the destination has not been reached on termination, we apply a failure cost as expressed in equation (9) and (10).

$$r = r_f + \sum_t r_t, \quad (9)$$

$$r_f = \begin{cases} 0, & \text{target reached (success),} \\ -c_f, & \text{target not reached (failure).} \end{cases} \quad (10)$$

We propose a topology-aware routing agent that combines an inductive message-passing GNN with a local per-edge Q-function, as illustrated in Fig. 2. The GNN encodes the current network graph into node embeddings that summarize the surrounding multi-hop topology, while the DQN scores the feasible next-hop candidates at the current node.

More specifically, each node  $i$  is initialized with a compact feature vector

$$\mathbf{h}_i^{(0)} = [\mathbb{I}\{i = S\}, \mathbb{I}\{i = T\}, \deg(i)], \quad (11)$$

while each edge  $e = (i, j)$  carries the attributes  $\mathbf{e}_{ij} = [L_{ij}, p_{ij}]$ . For message-passing layer  $\ell$ , node embeddings are updated as

$$\mathbf{m}_i^{(\ell+1)} = \sum_{j \in \mathcal{N}(i)} \phi_m(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ij}), \quad (12)$$

$$\mathbf{h}_i^{(\ell+1)} = \phi_u(\mathbf{h}_i^{(\ell)}, \mathbf{m}_i^{(\ell+1)}), \quad (13)$$

where  $\phi_m(\cdot)$  and  $\phi_u(\cdot)$  denote learnable message and update functions. After  $K$  layers, the topology-aware node representation is  $\mathbf{z}_i = \mathbf{h}_i^{(K)}$ . The encoder is shared across episodes and graph sizes, which enables inductive generalization to unseen topologies.

Given node embeddings, the per-edge DQN evaluates the feasible next hops of the current node. For a packet currently located at node  $i$ , a candidate next hop  $j \in \mathcal{N}(i)$ , and the corresponding edge  $e = (i, j)$ , we construct

$$\mathbf{x}_e = [\mathbf{z}_i \parallel \mathbf{z}_j \parallel L_e \parallel p_e \parallel \mathbb{I}\{j = T\}], \quad (14)$$

where  $\parallel$  denotes concatenation. Thus,  $\mathbf{x}_e$  explicitly contains the current-node context, the candidate-neighbour context, the link latency, the link PER, and a flag indicating whether the candidate is already the destination. A small multilayer perceptron (MLP) then outputs the action value

$$Q_\theta(s, a = j) = f_\theta(\mathbf{x}_e). \quad (15)$$

At decision time, the agent evaluates only the neighbours of the current node, which makes the local decision cost scale with the node degree rather than the total graph size.

For worst-case situations in which the learned policy produces an invalid route, e.g., due to a dead end or a loop, we invoke a deterministic fallback. Each link  $e$  is assigned the scalar cost  $c_e = w_L L_e - w_p \log(1 - p_e)$ , and Dijkstra's shortest-path algorithm is run on the weighted graph to recover a feasible route [6]. This fallback is used only when the DQN fails to terminate successfully; the episode-level failure cost in equation (9) therefore remains outside the edge weights.

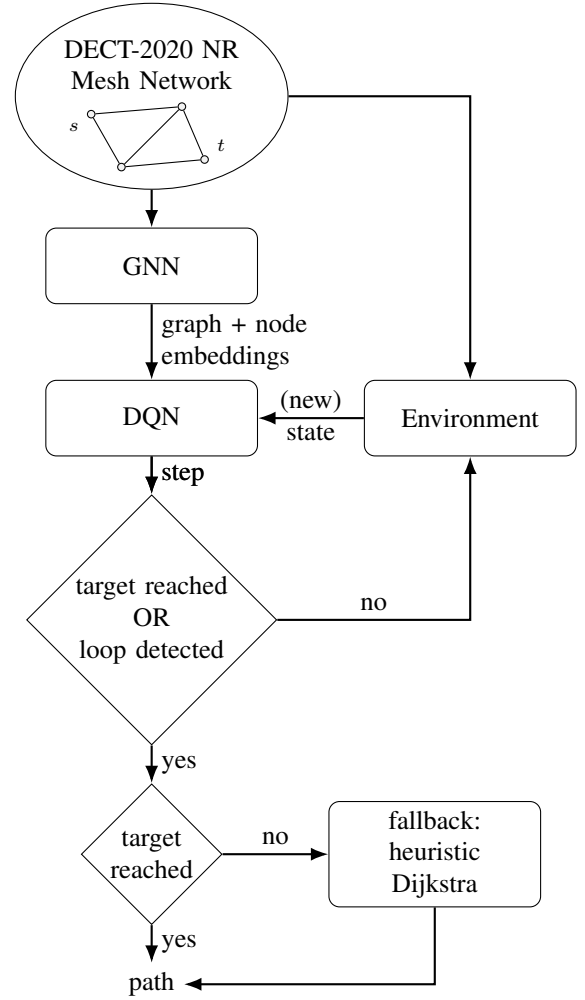


Fig. 2: Routing decision process with GNN-enhanced DRL and heuristic fallback.

### C. Dataset

Because no public dataset jointly provides measured DECT-2020 NR link-level latency and PER for the considered operating modes, we construct a link-metric dataset that maps radio conditions and configuration parameters to the edge attributes  $\{L_e, p_e\}$  used by the routing agent. We build upon our previous measurement work [9], which records SINR (and additional physical layer (PHY)/MAC metadata) for two-device DECT-2020 NR links under varying conditions such as distance, interference, and different DECT-2020 NR transmission configurations (e.g., carrier frequency  $f$ , numerology  $\mu$ , and Fourier scaling factor  $\beta$ ).

Each dataset row corresponds to one measurement snapshot and includes radio-quality indicators (e.g., SINR) and configuration parameters (e.g.,  $f$ ,  $\mu$ ,  $\beta$ ). In addition, we derive the user-plane latency  $L_e$  using the DECT-2020 NR-compliant frame-based latency model adopted in the paper [6]. To obtain PER values aligned with the standard PHY behaviour, we use our previous link-level simulation environment to generate PER-vs-SINR characteristics per configuration (e.g., numerol-

ogy) and interpolate  $p_e$  for each measured SINR sample [10]. The resulting dataset provides consistent paired labels ( $L_e, p_e$ ) for every edge instance used during graph generation.

TABLE I: DECT-2020 NR numerologies ( $\beta$  in relation to  $f$  and  $\mu$ ) [3].

| $f$  | $\mu = 1$    | $\mu = 2$ | $\mu = 4$ | $\mu = 8$ |
|------|--------------|-----------|-----------|-----------|
| 450  | {1, 2, 4}    | {1, 2}    | {1}       | –         |
| 1350 | {1, 2, 4, 8} | {1, 2, 4} | {1, 2}    | {1}       |
| 3700 | {1, 2, 4, 8} | {1, 2, 4} | {1, 2}    | {1}       |

#### D. Training

We train the routing agent using a double deep Q-learning variant with standard stabilizers [12]–[14], [20]. Specifically, we employ experience replay and update the Q-network using mini-batches sampled from a replay buffer to reduce temporal correlations between transitions. To mitigate overestimation bias, we maintain a separate target network whose parameters are updated periodically and used to compute the bootstrap targets. Exploration is realized via an  $\epsilon$ -greedy policy with a decaying exploration rate, enabling broad exploration early in training and more exploitative behaviour as learning progresses. Finally, we normalize rewards to improve numerical stability and learning dynamics, which is particularly important due to the log-reliability component in the URLLC-shaped reward.

To improve convergence and robustness, we adopt a curriculum over topology sizes. Training starts on smaller, simpler mesh networks and then increases the admissible graph size in stages, approximately doubling the node range after convergence on the current stage before progressing to the full training range [15], [19]. Domain randomization over link qualities and interference patterns further encourages policies that generalize beyond the training set. The GNN encoder uses a small number of message-passing layers with hidden dimension chosen to balance expressibility and compute cost. The per-edge Q-network is a shallow MLP. Training hyperparameters such as learning rate, batch size, replay buffer size, and target update period follow established Double Deep Q-Network (ddQN) practice [12], [13] and are tuned empirically for stability and convergence.

Because URLLC emphasizes rare-failure avoidance, we apply action masking during training and inference to remove neighbours that violate hard connectivity or loop-avoidance constraints. Reliability and latency are otherwise handled through the shaped reward in (7), which proved sufficient for the results reported in Section V.

#### IV. EVALUATION SETUP

We evaluate the proposed routing policy in a custom discrete-event simulator that reflects slot-based DECT-2020 NR operation and multi-hop forwarding in an industrial indoor scenario. The simulator couples four steps: topology generation, assignment of edge attributes  $\{L_e, p_e\}$  from the dataset in Section III-C, route computation, and packet-level accounting

of latency, delivery failures, and transmission overhead. For each graph realization, the gateway is fixed and the destination node is sampled from a connected randomly generated mesh topology. The reported end-to-end latency equals the sum of the path latency and the route-computation time. For flooding, the simulator additionally counts all replicated MAC-layer transmissions until delivery or termination.

The evaluation is therefore simulation-based; measured SINR samples are only used offline to parameterize the link metrics. No physical hardware setup is involved in this study, so Fig. 3 illustrates a representative generated mesh instance rather than a laboratory measurement setup. To mimic typical DECT-2020 NR deployments, we model the network as a set of interconnected clusters, where each cluster contains approximately 5 to 10 RDs. A single gateway operates in FT mode and provides backhaul access and downlink connectivity. Within and across clusters, RDs operating in FT/PT mode act as relays to extend coverage and interconnect clusters, whereas RDs operating in PT mode are primarily end nodes within a cluster. Network topologies are generated according to connected clustered random-graph models with randomized link conditions. Fig. 3 shows one representative graph instance used by the simulator.

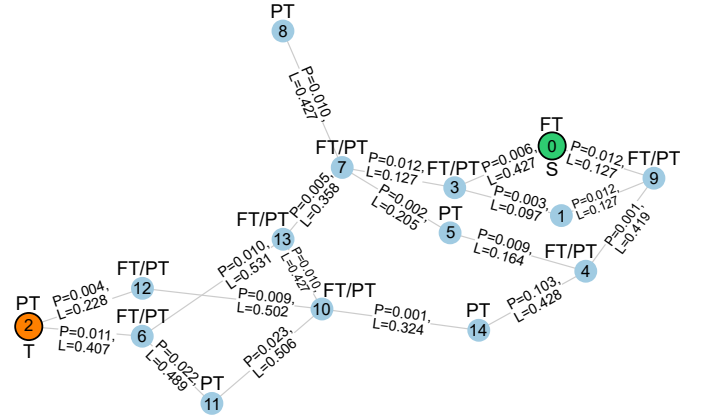


Fig. 3: Representative simulated DECT-2020 NR mesh graph with source  $S$ , destination  $T$ , node operating modes, and edge annotations for PER and latency.

All experiments were implemented in Python 3.11.9 using PyTorch 2.1.0 and PyTorch Geometric 2.3.1. NetworkX 3.1 was used for graph processing, Pandas 1.5.3 for data handling, NumPy 1.24.3 for numerical routines, TensorBoard 2.13.0 for logging, and Optuna 4.5.0 for hyperparameter tuning. Table II summarizes the most relevant simulator settings.

#### V. RESULTS AND DISCUSSION

Fig. 4–9 report the averaged performance of the proposed approach and the baselines as a function of node count. Each node-count point is obtained from  $10^4$  independently generated connected graphs. We compare against two baselines: (i) the DECT-2020 NR flooding procedure [4], and (ii) the latency-only centralized routing method of [6], which

computes the shortest path with Dijkstra's algorithm using expected per-hop latency.

TABLE II: Summary of the simulation setup.

| Parameter             | Value                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Topology size         | 2 to 101 nodes                                                                                                |
| Topology realizations | $10^4$ graph realizations per node-count point;<br>$10^5$ realizations for the fixed-61-node numerology sweep |
| Traffic direction     | Gateway-to-node downlink routing                                                                              |
| Node roles            | Gateway in FT mode; relays in FT/PT; end nodes in PT                                                          |
| Topology generation   | Connected clustered random graphs with randomized link conditions                                             |
| Link attributes       | $L_e$ from ETSI-compliant latency model; $p_e$ from measured SINR samples and linked PHY-level error curves   |
| Packet size           | 32 bytes                                                                                                      |
| Baselines             | DECT-2020 NR flooding and latency-only Dijkstra routing [6]                                                   |
| Reported metrics      | Calculation time, end-to-end latency, end-to-end <i>PER</i> , and total MAC-layer transmissions               |
| Software stack        | Python, PyTorch, PyTorch Geometric, NetworkX, Pandas, NumPy, Optuna, TensorBoard                              |

Fig. 4 shows that the route-computation time of the proposed framework scales better than the latency-only Dijkstra baseline once the topology becomes moderately large. The crossover occurs at approximately 48 nodes. This computation-time advantage directly contributes to the end-to-end latency in Fig. 5, where the proposed method becomes preferable on medium and large graphs. The same figure also highlights the substantial latency penalty of flooding, whose redundant transmissions increase contention, scheduling delay, and queueing overhead as the topology grows.

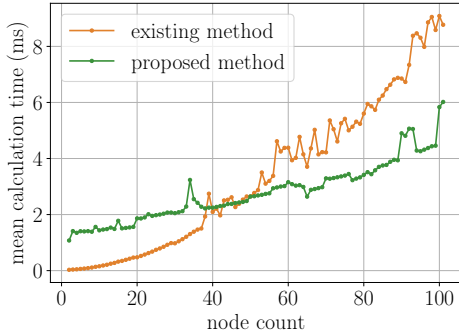


Fig. 4: Calculation time of Dijkstra (existing method) and this works proposed approach (GNN+DRL) vs. node count.

Fig. 6 highlights the reliability advantage of the proposed framework. Unlike the latency-only baseline, the proposed GNN+DRL agent explicitly includes link-level PER in the reward and therefore avoids routes with isolated unreliable hops. This matters because end-to-end success is multiplicative across hops, so one weak link can dominate the path failure probability. As a result, the learned policy often accepts a

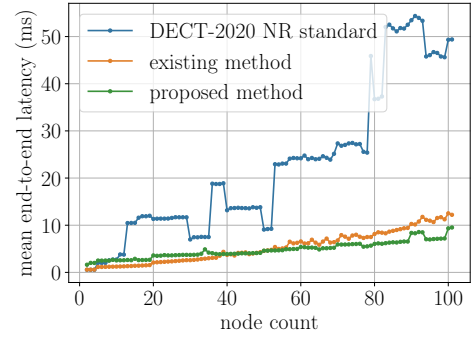


Fig. 5: End-to-end latency (calculation time + path latency) of existing and proposed method vs. node count.

small increase in hop count when this leads to markedly better end-to-end reliability, which explains the gap to the existing method in Fig. 6.

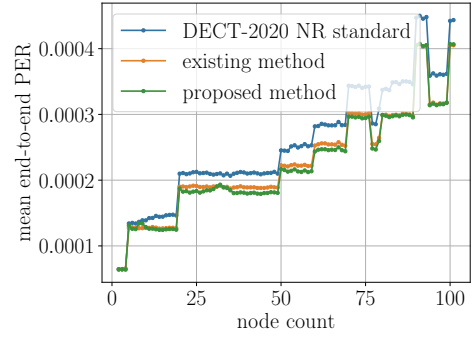


Fig. 6: End-to-end PER of chosen path of existing and proposed method vs. node count.

As the node count rises, the end-to-end latency rises as well, resulting in lower mean throughput. This change can be seen in Fig. 7. It visualizes mean throughput for 1000 packets of size 32 bytes sent at each node count. After the turning point, the proposed method outperforms the existing one.

To quantify network load, we measure the total number of *MAC-layer transmissions* triggered by delivering a single source packet on each graph. For the existing method and the proposed solution, each hop corresponds to exactly one packet transmission; hence, the total number of transmitted packets equals the path length (i.e., hop count). In contrast, flooding replicates the packet at multiple relays: every node that forwards the packet contributes additional transmissions, so the total number of transmitted packets is the sum of all broadcast/forwarding packets until the target is reached (or a hop/TTL limit expires). Fig. 8 therefore visualizes the substantially higher transmission overhead of flooding, which translates into higher channel occupancy, interference, and congestion in dense mesh networks.

Fig. 9 zooms into the difference between the existing and proposed methods only. The proposed policy uses slightly

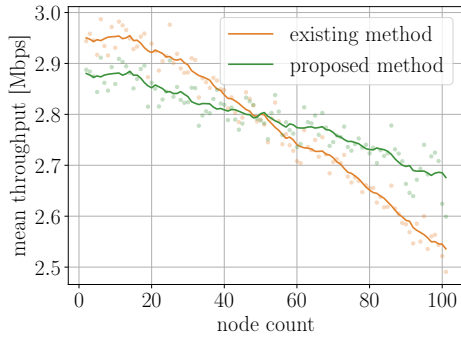


Fig. 7: End-to-end throughput of chosen path of existing and proposed method for 1000 packets of size 32 bytes vs. node count (smoothed by using mean window size 10).

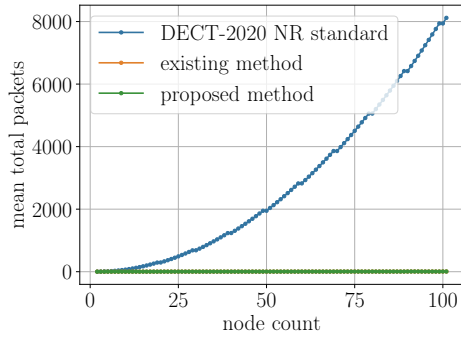


Fig. 8: Number of total packets sent of DECT-2020 NR standard (flooding), existing and proposed method vs. node count (proposed method is overlaying the existing method).

more hops on average, but this gap remains small and tightly bounded across topology sizes. In other words, the agent trades a modest increase in path length for lower route-computation time and markedly better reliability instead of reverting to flooding-like transmission overhead.

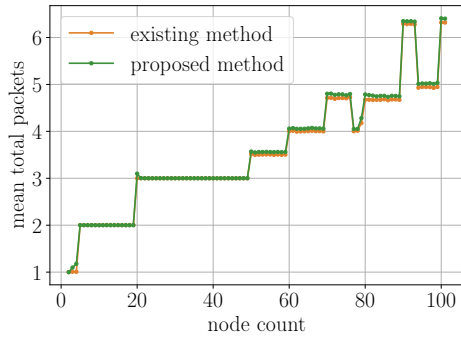


Fig. 9: Number of total packets sent of existing and proposed method vs. node count.

1) *DECT-2020 NR Transmission configuration*: Fig. 10 and 11 focus on a fixed node count of 61, i.e., well beyond the crossover region observed in Fig. 4. These experiments use  $10^5$  independently generated graphs. The parameter sweep over the numerology  $\mu$  reveals how the DECT transmission configuration influences the routing trade-off: since  $\mu$  determines the subslot duration and therefore the transmission-time granularity, it directly affects the hop latency and consequently the end-to-end latency shown in Fig. 10. Moreover, end-to-end latency includes calculation time overhead; the proposed solution scores by up to 19.9% better than the existing method. The performance gap is most pronounced at lower numerologies ( $\mu \in \{1, 2\}$ ), where the longer subslot duration amplifies the relative benefit of faster topology-aware path computation.

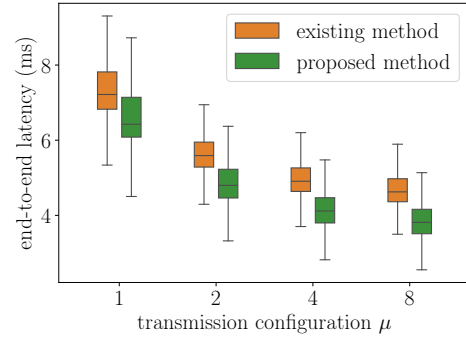


Fig. 10: End-to-end latency (calculation time + path latency) of existing and proposed method vs. subcarrier scaling factor on a graph of 61 nodes.

In Fig. 11, the proposed solution lowers end-to-end PER by up to 10% compared with the existing method while also showing visibly tighter dispersion across the tested numerologies. The reliability gain stems directly from the URLLC-shaped reward: by penalizing unreliable hops through the  $\log(1 - p_e)$  term, the agent avoids weak links even when they would slightly shorten the path. This confirms that explicitly accounting for reliability during path selection yields a practical benefit beyond the latency-only baseline.

#### A. URLLC Trade-Off and Scalability

Across the considered DECT-2020 NR mesh sizes, the proposed GNN+DRL agent achieves a more favourable reliability-latency trade-off than both baselines. For a given latency budget, it delivers higher end-to-end success probability; for a target reliability level, it reaches lower latency than flooding and lower failure probability than the latency-only routing baseline. Qualitatively, the policy tends to choose paths with slightly more hops but more reliable constituent links, which is consistent with the reward design and the reliability improvements observed in Figs. 6 and 11.

From a computational perspective, the local decision cost of the learned policy scales mainly with the current node degree because only feasible neighbours are scored once node embeddings are available. This explains the improved scaling

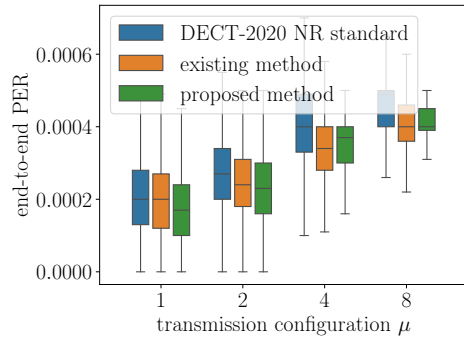


Fig. 11: End-to-end PER of chosen path of DECT-2020 NR standard, existing and proposed method vs. subcarrier scaling factor on a graph of 61 nodes.

trend in Fig. 4 and suggests that topology-aware learning is practical for larger DECT-2020 NR mesh networks.

## VI. CONCLUSION AND FUTURE WORK

This paper presented a GNN-enhanced DRL framework for URLLC-oriented routing in DECT-2020 NR mesh networks. We formulated next-hop selection as an MDP aligned with the DECT-2020 NR system model, introduced a URLLC-shaped reward that captures the interplay between reliability and latency and proposed a routing agent that combines an inductive GNN encoder with a local per-edge DQN head. Simulation results indicate that the proposed approach improves the reliability-latency trade-off compared to classical routing baselines and reduces network load related to flooding. The analysis of scalability and generalization further suggests that topology-aware learning is a viable building block for future URLLC-capable mesh systems. Future work includes coupling routing with joint power/slot/channel allocation, integrating queue and scheduling information into the graph representation, and evaluating the approach on hardware platforms representative of practical DECT-2020 NR deployments. Another important direction is to explore tighter safety guarantees and formal verification for learning-based routing policies in safety-critical industrial environments.

## ACKNOWLEDGMENT

The work in this paper was part of the CELTIC-NEXT project USWA and received funding from the Federal Ministry of Education and Research of Germany (BMBF, funding code: 16KISK254). We would like to thank the funding organization for the financial support.

## REFERENCES

- [1] European Telecommunications Standards Institute, “DECT-2020 New Radio (NR): Part 1: Overview; Release 1,” ETSI, Tech. Rep. ETSI TS 103 636-1 V1.3.1, Dec. 2021.
- [2] —, “DECT-2020 New Radio (NR): Part 4: MAC Layer; Release 1,” ETSI, Tech. Rep. ETSI TS 103 636-4 V1.4.1, 2023.
- [3] —, “DECT-2020 New Radio (NR): Part 3: Physical Layer; Release 1,” ETSI, Tech. Rep. ETSI TS 103 636-3 V1.1.1, Jan. 2024.

- [4] —, “DECT-2020 New Radio (NR): Part 5: DLC and Convergence Layers; Release 1,” ETSI, Tech. Rep. ETSI TS 103 636-5 V1.6.1, 2025.
- [5] R. Kovalchukov, D. Moltchanov, J. Pirskanen, J. Sae, J. Numminen, Y. Koucheryavy, and M. Valkama, “DECT-2020 New Radio: The Next Step toward 5G Massive Machine-Type Communications,” *IEEE Communications Magazine*, vol. 60, no. 6, pp. 58–64, Jun. 2022.
- [6] A. B. Asif, H. Ahmad, S. Preihs, and J. Peissig, “URLLC-Based Multi-Hop Communication Using DECT-2020 NR,” in *Proc. 2025 IEEE Wireless Communications and Networking Conference (WCNC)*, 2025, pp. 1–7.
- [7] A. Anttonen, P. Karhula, M. Lasanen, and M. Majanen, “Enabling Massive Machine-Type Communications with DECT-2020 Standard: A System-Level Performance Study,” VTT Technical Research Centre of Finland, Tech. Rep. VTT-R-00367-21, 2021.
- [8] European Telecommunications Standards Institute, “Final Evaluation Report on DECT-2020 NR,” ETSI, Tech. Rep. ETSI TS 103 810-1 V1.1.1, Nov. 2021.
- [9] M. Waßmann, A. Poets, J. Peissig, and J. Pilz, “Performance of DECT-2020 NR in an industrial environment for varying RF bands,” in *2024 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*, 2024, pp. 610–615.
- [10] M. Penner, M. Nabeel, and J. Peissig, “URLLC Performance Evaluation of IMT-2020 Candidate Technology: DECT-2020 New Radio,” in *2021 IEEE 94th Vehicular Technology Conference (VTC2021-Fall)*, 2021, pp. 1–7.
- [11] H. Ahmad, A. B. Asif, J. Peissig, and M. Penner, “MAC Protocol for Reducing Outage Probability in DECT-2020 New Radio,” in *Proc. 2024 20th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*, 2024, pp. 42–49.
- [12] D. T. Hoang, N.-N. Dao, D. Niyato, and E. Hossain, *Deep Reinforcement Learning for Wireless Communications and Networking: Theory, Applications and Implementation*. John Wiley & Sons, 2023.
- [13] Q. He, Y. Wang, X. Wang, W. Xu, F. Li, K. Yang, and L. Ma, “Routing Optimization With Deep Reinforcement Learning in Knowledge Defined Networking,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 2, pp. 1444–1455, 2024.
- [14] Y. Song, M. Ni, and J. Liu, “Deep Reinforcement Learning for Network Routing Optimization: A Systematic Survey,” *Neurocomputing*, vol. 666, p. 132263, 2026.
- [15] X. Zhang, H. Zhao, J. Xiong, X. Liu, H. Yin, X. Zhou, and J. Wei, “Decentralized Routing and Radio Resource Allocation in Wireless Ad Hoc Networks via Graph Reinforcement Learning,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 10, no. 3, pp. 1146–1159, 2024.
- [16] Y. Dai, L. Lyu, N. Cheng, M. Sheng, J. Liu, X. Wang, S. Cui, L. Cai, and X. Shen, “A Survey of Graph-Based Resource Management in Wireless Networks—Part II: Learning Approaches,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 11, no. 4, pp. 2101–2121, 2025.
- [17] Z. Huang, Z. Wang, and Z. Han, “Graph Neural Network Empowered Wireless Communications: Fundamentals, State-of-the-Art, Challenges, and Opportunities,” *IEEE Wireless Communications*, vol. 32, no. 5, pp. 162–168, 2025.
- [18] Y. Lu, Y. Li, R. Zhang, W. Chen, B. Ai, and D. Niyato, “Graph Neural Networks for Wireless Networks: Graph Representation, Architecture and Evaluation,” *IEEE Wireless Communications*, vol. 32, no. 1, pp. 150–156, 2024.
- [19] M. Ding, Y. Guo, Z. Huang, B. Lin, and H. Luo, “GROM: A Generalized Routing Optimization Method with Graph Neural Network and Deep Reinforcement Learning,” *Journal of Network and Computer Applications*, vol. 229, p. 103927, 2024.
- [20] B. Ghoghgh and A. Ghodsi, “Reinforcement Learning: Tutorial and Survey,” *OSF Preprints*, 2024.